

Mac 2.0 Hypercube walkthrough

Fuel cost and tax scenario matrix

Build a two-parameter experiment, run its 25 cases, and examine how each combination changes a selected output relative to one common baseline. The example follows Fuel Cost and Tax in the Mac video.

[Hypercube video](#)

[All Mac videos](#)

Before you begin

Install PlanRVA with its paired inputs and verify the Mac runtime. Keep Docker Desktop running. Check Settings → Resources and available disk space before creating a matrix: every case is a complete model run with retained results.

Five fuel-cost values multiplied by five fuel-tax values produce 25 cases. The common baseline adds one run if it is not already available. Target year and shared geography select the affected input rows; they do not add matrix dimensions.

Create a Hypercube project

1. Open Hypercube → Build. Create a dedicated Hypercube project named Fuel Cost and Tax using PlanRVA and its paired Input Library. Use a fresh baseline for this example.

The screenshot shows the VisionEval Workbench interface. At the top, there's a header with "VISIONEVAL WORKBENCH" and "Build, run, and understand scenarios". On the right, there are buttons for "Dark mode" and "VisionEval runtime ready". Below the header is a navigation bar with five tabs: "1 Explore", "2 Create", "3 Run", "4 Compare", and "5 Hypercube". The "5 Hypercube" tab is selected.

Under the "5 Hypercube" tab, there's a section titled "LARGE SCENARIO EXPERIMENTS" with the heading "Hypercube". Below this, it says "Build, review, run, and analyze a bounded parameter matrix as one workflow." There are five sub-tabs: "Build", "Review", "Run", "Analyze", and "Export". The "Build" sub-tab is selected.

The "Build" sub-tab contains a "PROJECT SETUP" section. It has a "Project name" field with the text "Fuel Cost and Tax" and a "Model package" dropdown menu showing "PlanRVA - 51 CSVs". There is a "Create Hypercube project" button. Below this, there's a "SCENARIO MATRIX" section with the heading "Hypercube". It says "Create every combination across selected input-column ranges. Generated scenarios always start from baseline model-package values." There is a "Hypercube project" dropdown menu with the text "Choose a Hypercube project" and a note "The project name is also the matrix name." Below this, there's a "Parameter axes" section with the text "Each row becomes one matrix axis. The final scenario count is the product of every axis value count." and an "+ Add parameter" button.

Figure 1. A Hypercube project keeps its matrix and common baseline together.

Use a dedicated Hypercube project rather than adding a large matrix to your standard comparison project. The same installed PlanRVA package can supply both projects without changing the source package.

Define the two parameter axes

2. Add `azone_fuel_power_cost.csv` and select `FuelCost.2024`. Choose Increase by %, Start 10, End 50, and Interval 10.
3. Add `azone_veh_use_taxes.csv` and select `FuelTax.2024` with the same operation and range. Set shared target year 2045 and geography All matching rows.

The screenshot displays the VisionEval Workbench interface for configuring parameters. It features two main sections for parameter definition, each with a table of settings and a warning message.

Input file	Numeric column	Operation	Start	End	Interval
<code>azone_fuel_power_cost.csv</code>	<code>FuelCost.2024</code>	Increase by %	10	50	10

Coordinates, protected fields, and linked shares are excluded. Share compositions must be edited together.

Input file	Numeric column	Operation	Start	End	Interval
<code>azone_veh_use_taxes.csv</code>	<code>FuelTax.2024</code>	Increase by %	10	50	10

Coordinates, protected fields, and linked shares are excluded. Share compositions must be edited together.

Shared scope
This scope is applied to every matrix combination. It does not add a location dimension or aggregate values to the MPO.

Shared target year	Shared geography	Locations
2045	All matching rows	All matching rows

Shared target year selects which dated CSV rows receive matrix changes; it does not change the model's configured years.

Loading every selected parameter file before resolving shared scope...

Buttons: Saved, Preview Matrix, Generate Scenarios

Configure the parameters, then preview the matrix.

Figure 2. Each parameter has five values and shares the same target year and scope.

The values are 10, 20, 30, 40, and 50 percent increases from the baseline. The interval is the step size, not the number of cases. Confirm the saved draft before previewing it.

Preview the matrix and resources

4. Choose Preview Matrix. Check both value lists, the case count, affected rows, and resource estimate before generating scenarios.

The screenshot shows the VisionEval Workbench interface. At the top, there is a 'Saved' button and two main buttons: 'Preview Matrix' and 'Generate Scenarios'. The main content area is divided into several sections:

- Fuel Cost and Tax:** A matrix visualization showing a 5x5 grid of blue dots. Below it, the text 'FuelCost.2024 (5)' and '5 x 5 = 25 scenarios' is displayed.
- Preview Matrix (Red Box 1):** This section contains three input fields:
 - 25 scenarios:** A text input field.
 - 2 parameter axes:** A text input field.
 - 16 changed cells per case:** A text input field.
- Example combinations:** A list of three cases:
 - Case 1: FuelCost.2024 = 10; FuelTax.2024 = 10
 - Case 2: FuelCost.2024 = 10; FuelTax.2024 = 20
 - Case 3: FuelCost.2024 = 10; FuelTax.2024 = 30
- Resource estimate (Red Box 2):** This section provides details on execution, memory, and disk usage.
 - Execution:** 5 waves at 6 parallel, about 57 minutes.
 - Peak Docker memory:** 15–21 GB, While 6 runs are active.
 - Retained disk:** about 7.1 GB, Datastores only - no optional full CSV trees.

At the bottom, there is a note: 'Based on 2 comparable completed runs. Based on 2 comparable retained results.' and a link: 'Open Settings → Resources'.

Figure 3. The preview confirms 25 combinations, two axes, and 16 changed cells per case.

1 Confirm case and axis counts. 2 Review execution, memory, and disk estimates.

This example changes all eight Azone rows in each file. Locations are shared scope, not additional cases. The preview estimated five execution waves at six concurrent runs. Estimates depend on measured runs and your computer; the video's duration is not a guarantee.

Generate and review scenarios

5. Choose Generate Scenarios only after checking the preview. Saving the Build draft does not generate the cases automatically.

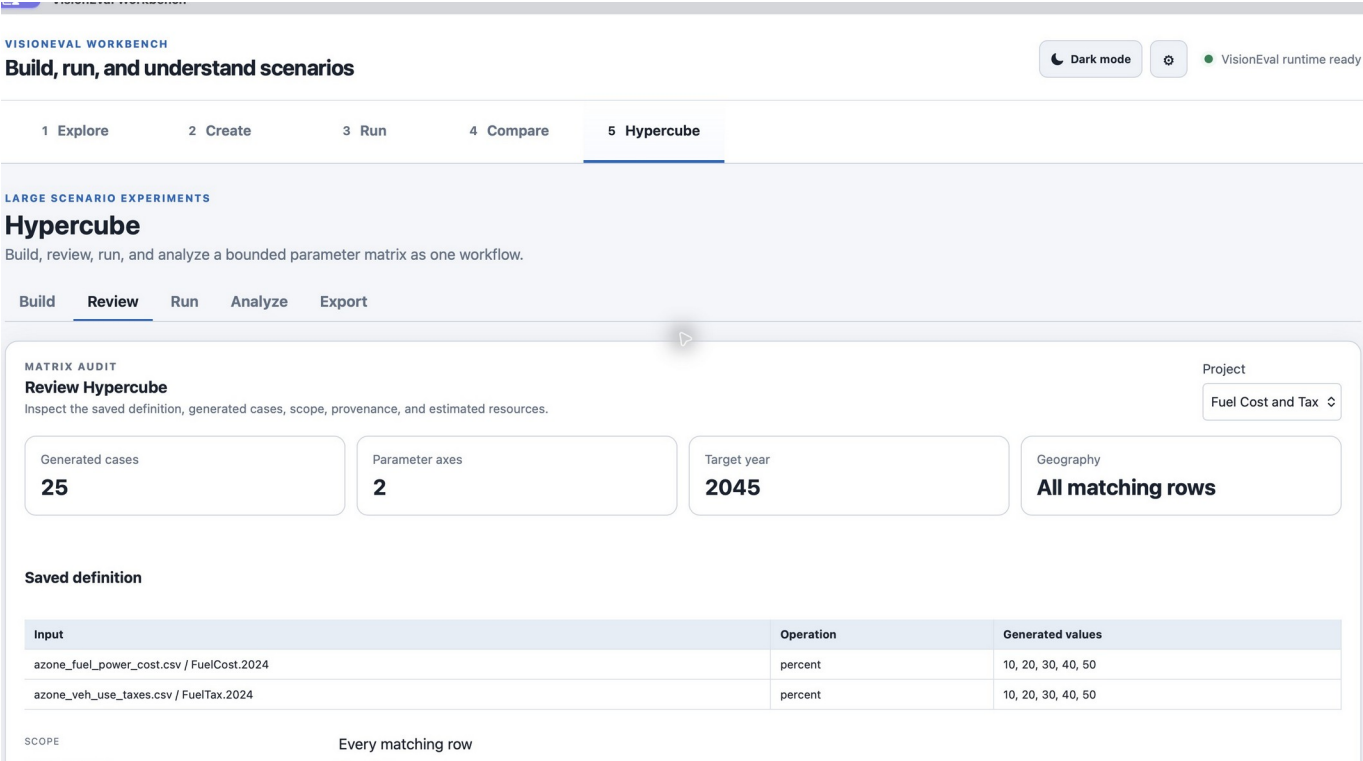


Figure 4. Review the generated matrix definition before submitting cases.

6. Open Review and check the definition and provenance. Once a case has been submitted to Run or produced a result, the matrix is locked. Create a new Hypercube project if you need a different experiment.

Run the matrix

7. Open Hypercube → Run and choose Run Missing. Workbench queues the common baseline if needed and cases without completed results. Keep Docker and Workbench open until execution finishes.

The screenshot displays the VisionEval Workbench interface, specifically the 'Hypercube' section under 'LARGE SCENARIO EXPERIMENTS'. The top navigation bar includes '1 Explore', '2 Create', '3 Run', '4 Compare', and '5 Hypercube'. Below this, the 'Hypercube' title is followed by the subtitle 'Build, review, run, and analyze a bounded parameter matrix as one workflow.' and a sub-navigation bar with 'Build', 'Review', 'Run' (selected), 'Analyze', and 'Export'.

The main content area is titled 'BATCH EXECUTION' and 'Run Hypercube', with a note: 'Run only unfinished work and monitor the matrix as one batch.' A 'Project' dropdown menu is set to 'Fuel Cost and Tax'. The execution status is shown in five boxes: 'Successful' (26), 'Missing' (0), 'Failed' (0), 'Waiting' (0), and 'Running' (0). Below these are three buttons: 'Run Missing' (blue), 'Retry Failed' (light blue), and 'Stop This Hypercube' (red).

A progress bar for 'Fuel Cost and Tax' shows '26 of 26 complete' at 100%. Below this, a summary table displays: 'Execution waves' (5), 'Elapsed' (55m 7s), 'Concurrency' (6), 'Estimated remaining' (Complete), and 'Status' (Complete). A note at the bottom states: 'All generated cases are complete. More parallel runs can finish the Hypercube faster by reducing execution waves, provided Docker has enough memory. Too much parallelization for the available memory can slow or fail runs. Results retain Datastore only. Hypercube runs do not generate the optional full CSV trace.' A link 'Open Settings → Resources' is at the bottom right.

Figure 5. All 25 cases and the common baseline completed successfully.

The documentation example completed in about 55 minutes with concurrency six. This is an observed run, not a promised duration. Retry Failed is for failed cases; successful cases do not need routine reruns. Stop This Hypercube stops this batch, not unrelated batches.

Find changes across outputs first

8. Open Hypercube → Analyze and select Fuel Cost and Tax. Scroll to Find All Changes. Set its Year to 2045 and Aggregation to Median, then choose Find All Changes before exploring a single variable. Wait for the scan to finish or use Cancel.

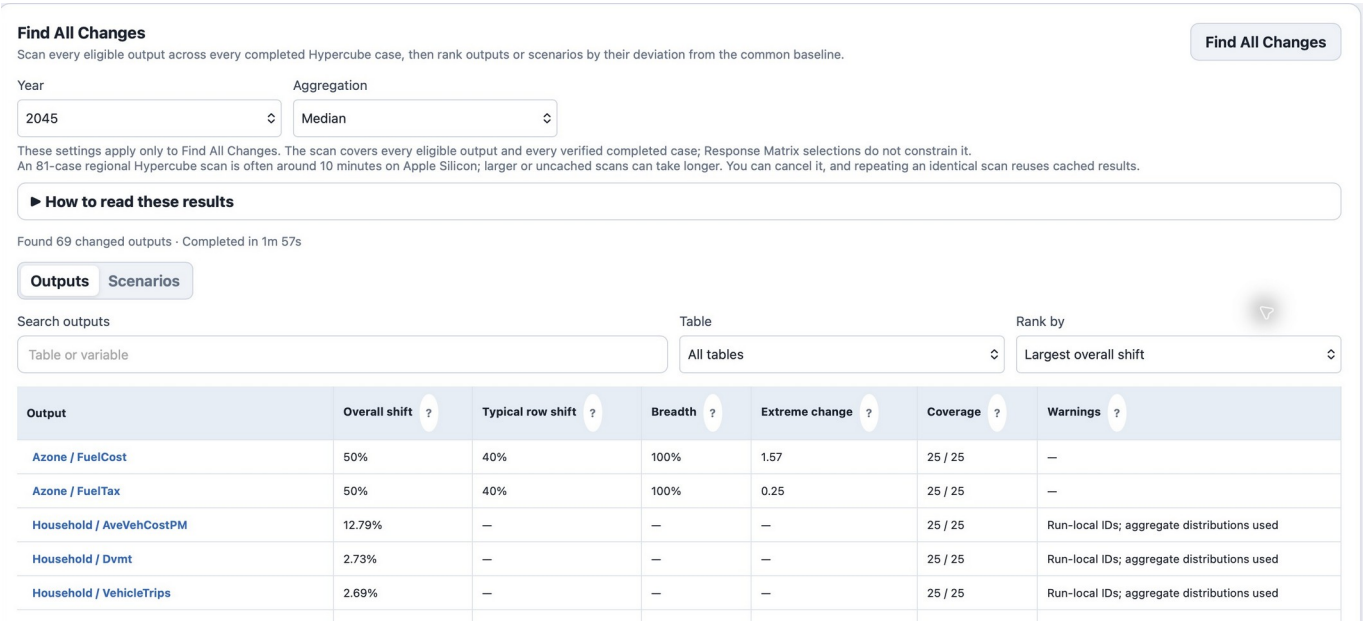


Figure 6. Find All Changes ranks output sensitivity across completed cases.

These Year and Aggregation controls apply only to Find All Changes. The scan covers eligible outputs across every verified completed case; the single-output Response Matrix selections do not constrain it. Review Outputs to find affected variables and Scenarios to compare case sensitivity. Select an output to inspect its scenario ranking.

The ranking measures deviation from the common baseline. It does not identify the best policy. Inspect the units, reference value, output, and scenario before interpreting a large change. Repeating an identical scan can reuse cached results.

Explore a variable in the response matrix

9. After reviewing Find All Changes, return to the single-output analysis controls to explore a variable in detail. For this example, select 2045, Household, AveVehCostPM, Percent change from baseline, Mean, and All locations. Keep FuelCost.2024 on the X axis and FuelTax.2024 on the Y axis. Choose Update Analysis.

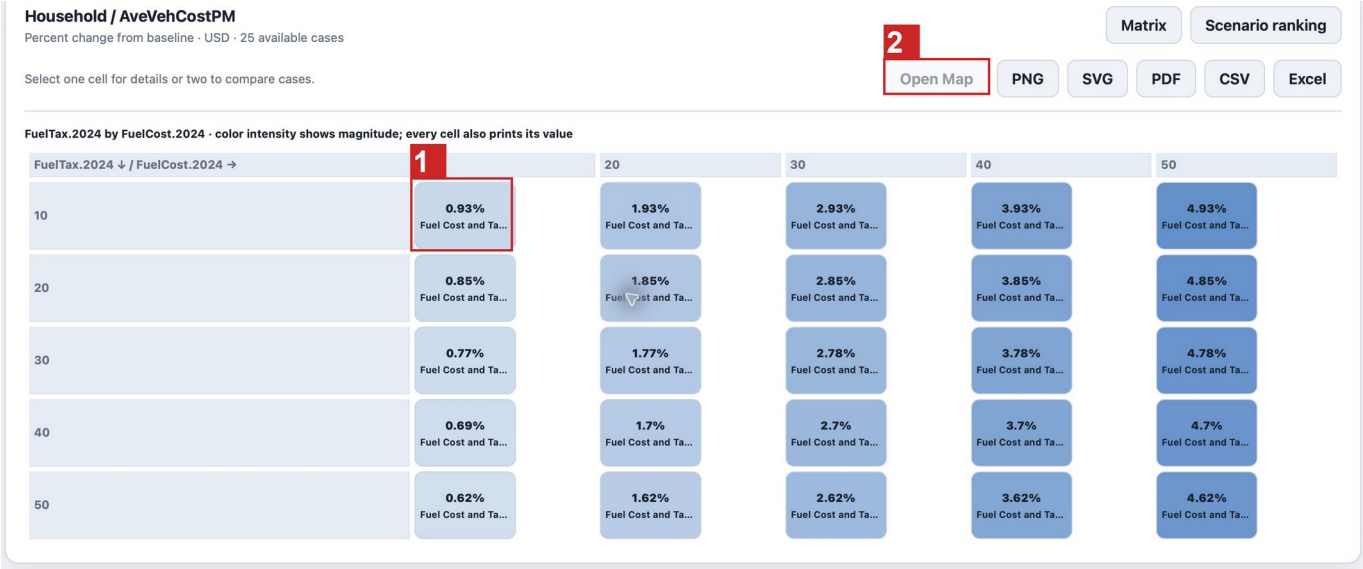


Figure 7. Select a matrix cell, then use Open Map to examine that case spatially.

1 Select a case cell. 2 Open Map becomes available after selecting a case.

Each cell represents one fuel-cost and fuel-tax combination. Positive percent change means an increase in the selected output, not automatically a better result. Mean, Median, and Sum answer different questions; choose the aggregation deliberately.

Inspect and export the analysis

10. Select one matrix cell for case details or two cells to compare their values. Use Scenario ranking when a sorted list is easier to read.

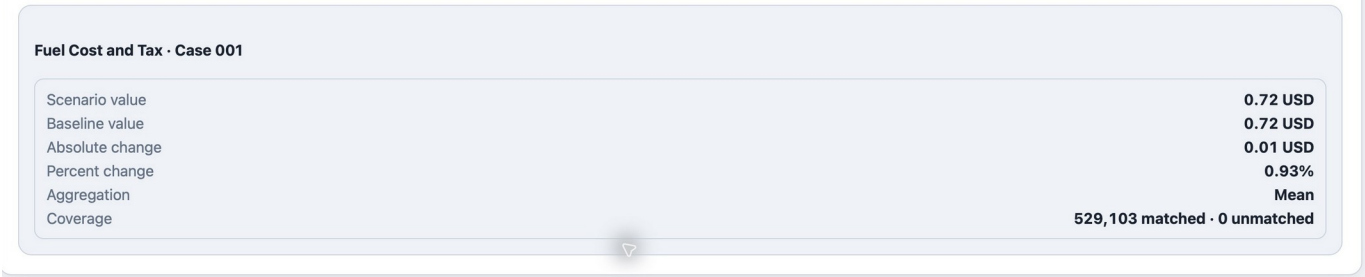


Figure 8. Case details show the selected case and calculated comparison.

To see a case on a map, select its cell in the Response Matrix and choose Open Map in the toolbar above the matrix. Workbench opens Compare → Map Visualization with the common baseline, selected case, table, and variable. Check the year, geography level, aggregation, and displayed measure, then choose Generate Map. If two cells are selected, Open Map uses the most recently selected case against the common baseline, not a map of the difference between those two cases.

11. Return to Hypercube → Analyze for chart exports. Use PNG, SVG, or PDF for a visual, and CSV or Excel for data. Save the file, open it, and check the axis labels and calculation. These controls export the displayed analysis, not the complete model result.

Export a case package

12. Open Hypercube → Export. Select up to three completed cases per batch and start the export. Workbench queues temporary output-CSV generation and offers a save dialog for each ZIP.

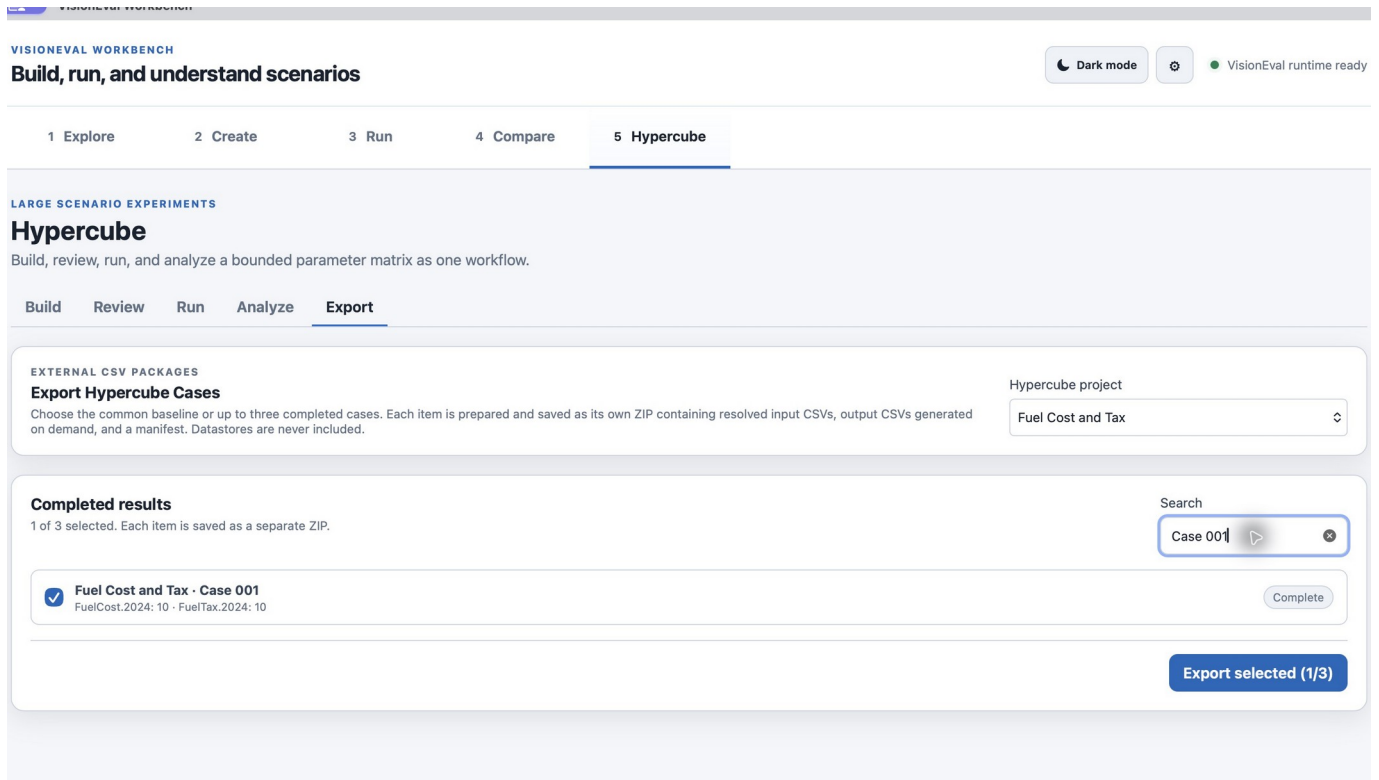


Figure 9. Case packages export resolved input and generated output CSV files.

Case packages are different from chart exports. Keep the original workspace and authoritative Datastores; an exported case ZIP is not a complete workspace backup. Hypercube runs retain Datastores without persistent full CSV trees, and the analysis cache can be rebuilt when needed.